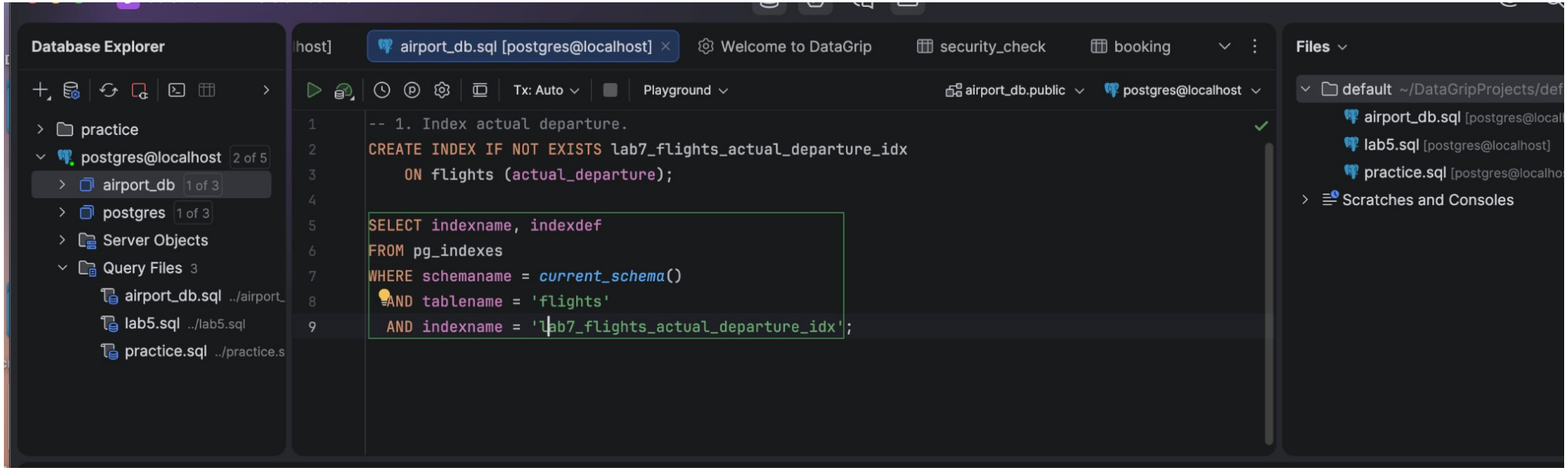
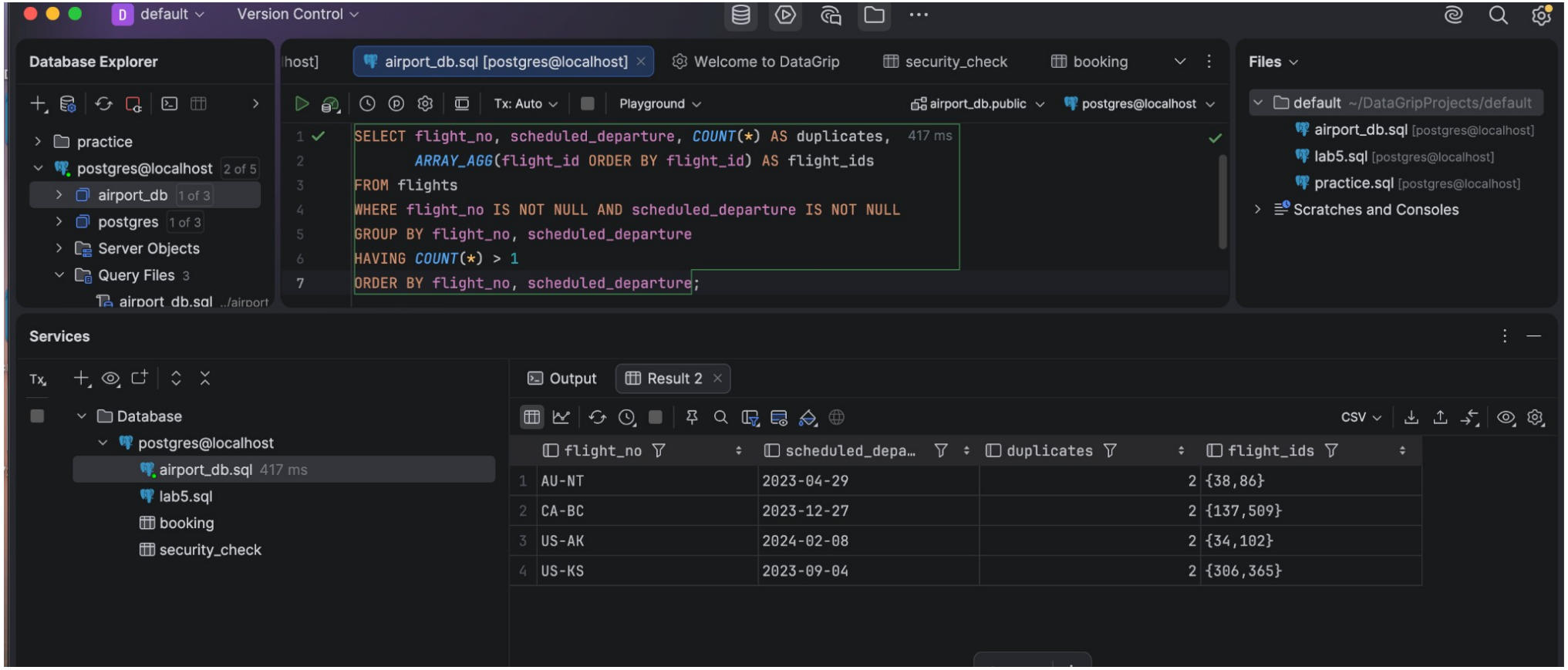
Task 1 Index on actual departure



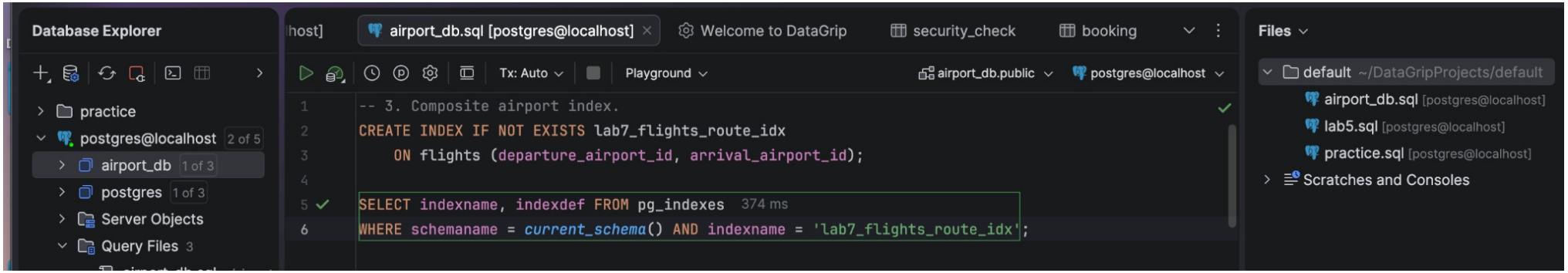
Result and explanation The SQL creates an index on flights.actual_departure. The catalog query checks its definition by name.

Task 2 Unique index on flight number and date



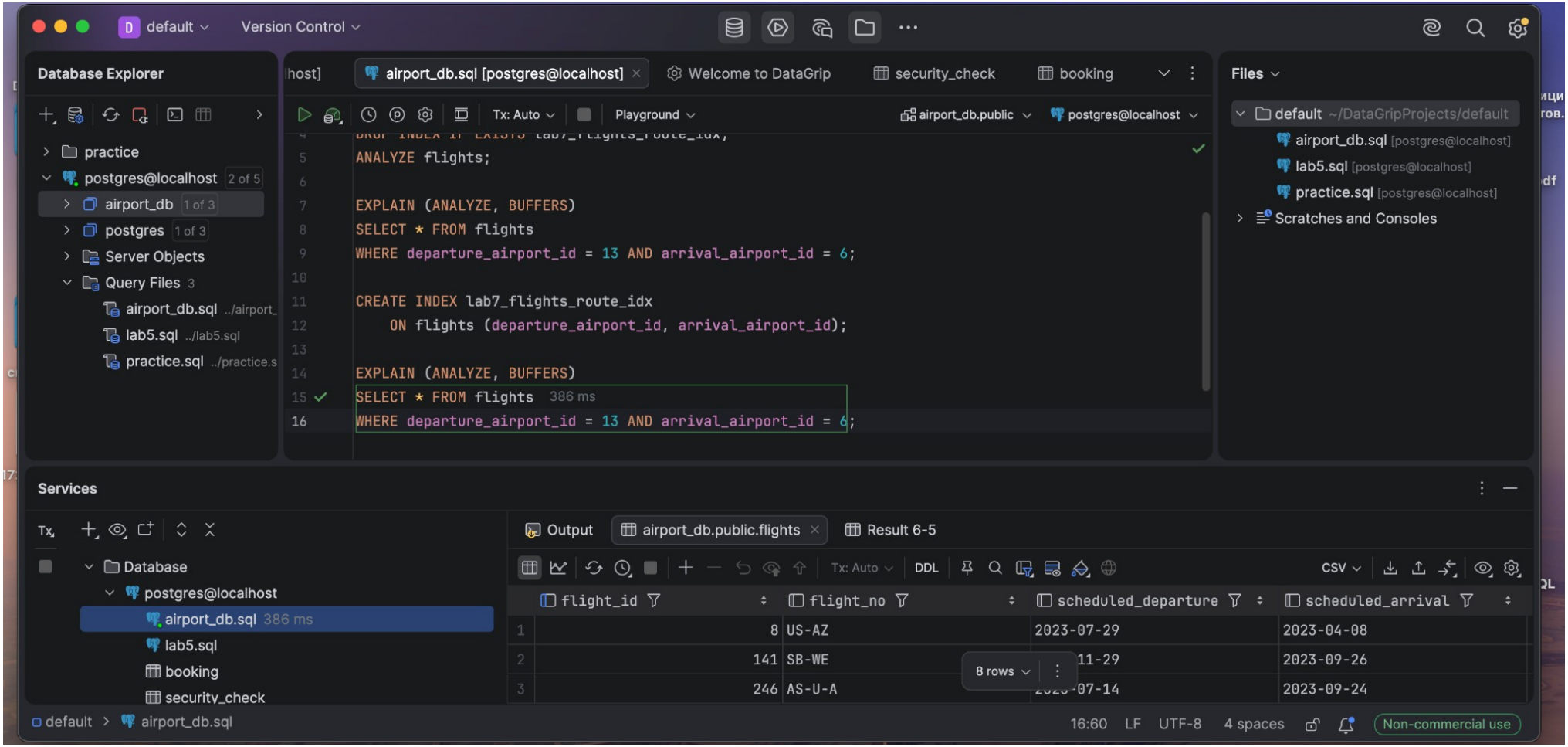
Result and explanation The duplicate check shows four repeated pairs in this run. PostgreSQL cannot create a unique index on (flight_no, scheduled_departure) until those conflicts are resolved. This screenshot documents the duplicate check.

Task 3 Composite route index



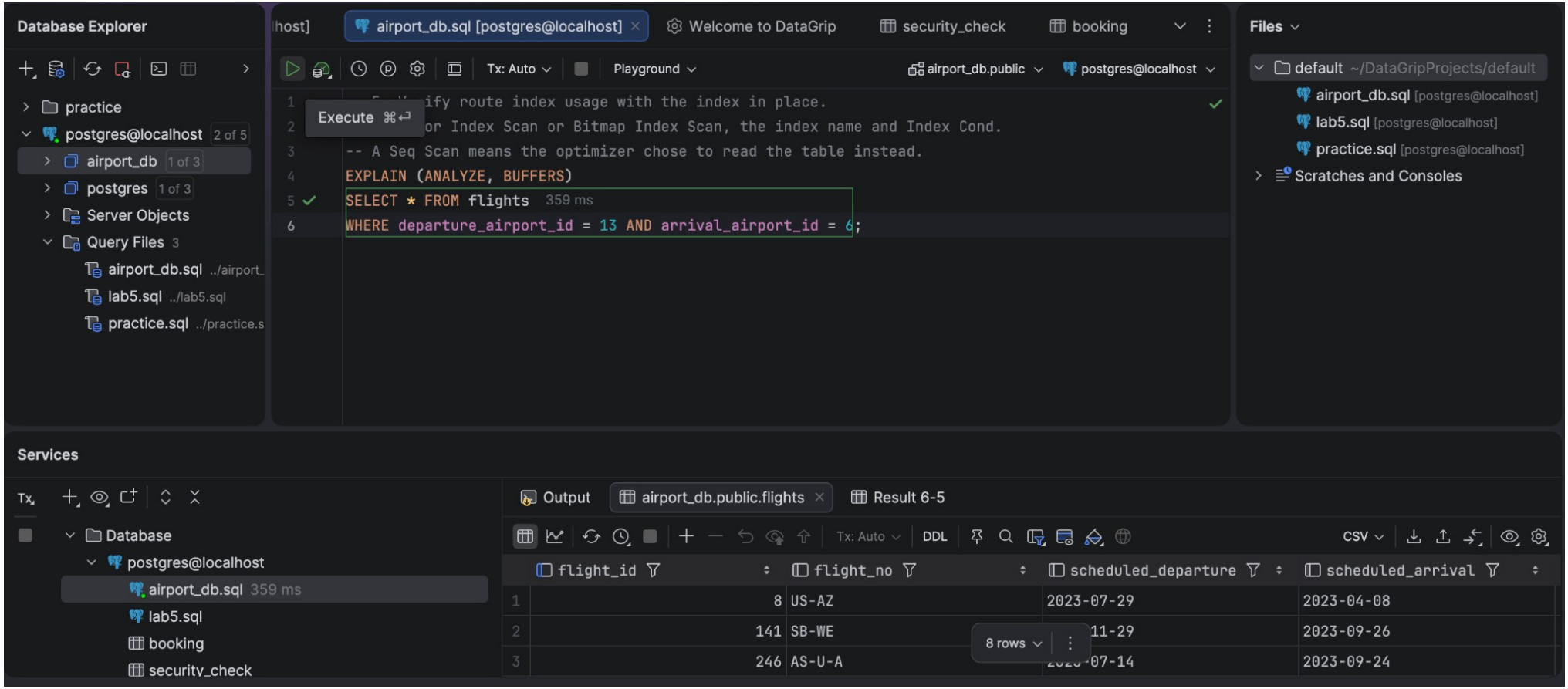
Result and explanation The index stores departure_airport_id first and arrival_airport_id second. The catalog query checks the definition of lab7_flights_route_idx.

Task 4 Route query before and after indexing



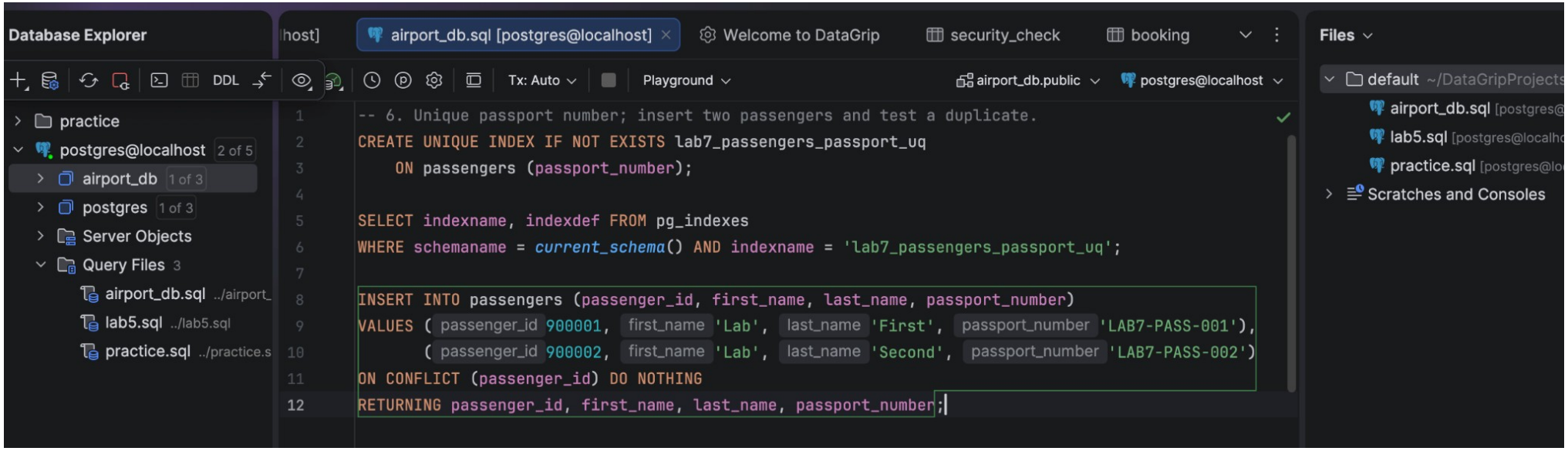
Result and explanation The same route filter is used before and after creating the index. The result grid shows eight matches. To measure the effect of the index, compare the two plans' Execution Time and Buffers; those measurements are not visible here.

Task 5 Route index usage



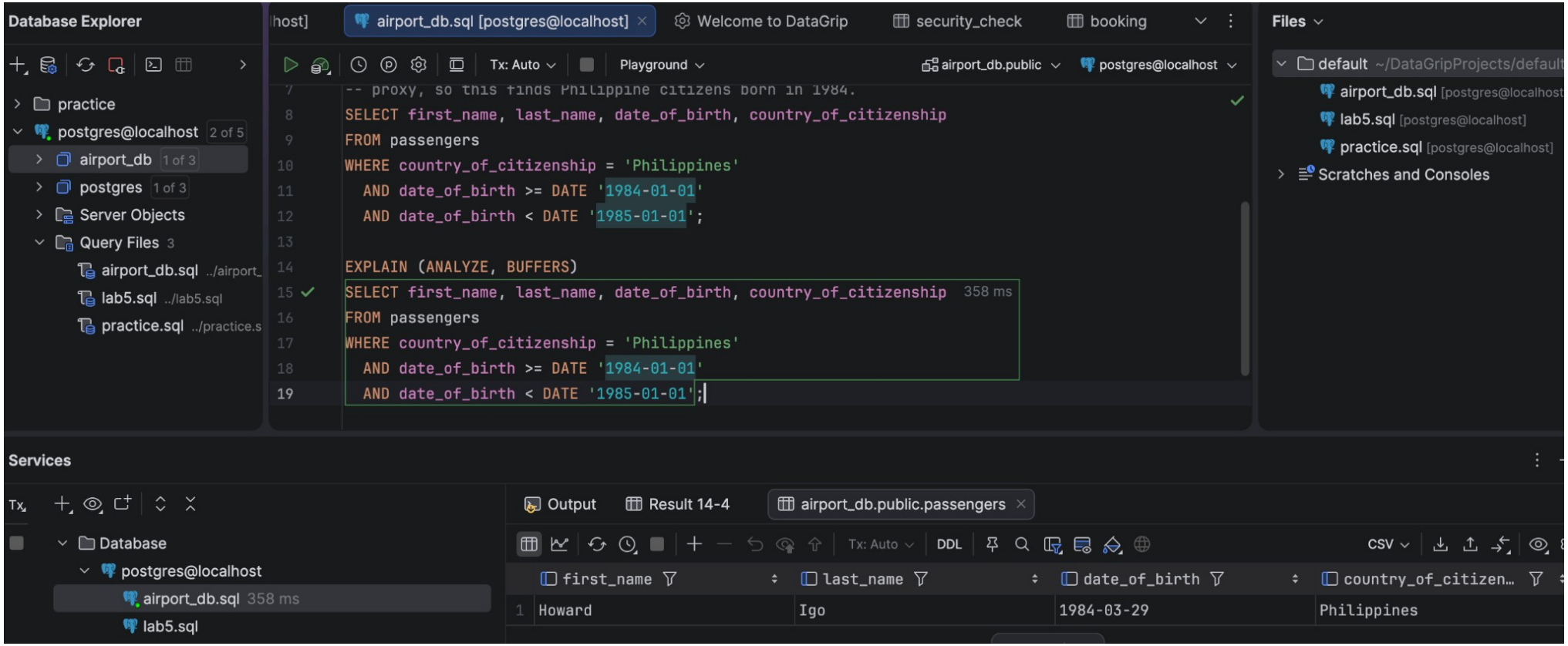
Result and explanation The filter selects flights from airport 13 to airport 6 and returns eight rows. Index usage is verified in the EXPLAIN plan by looking for lab7_flights_route_idx; the screenshot shows the data grid.

Task 6 Unique passport numbers



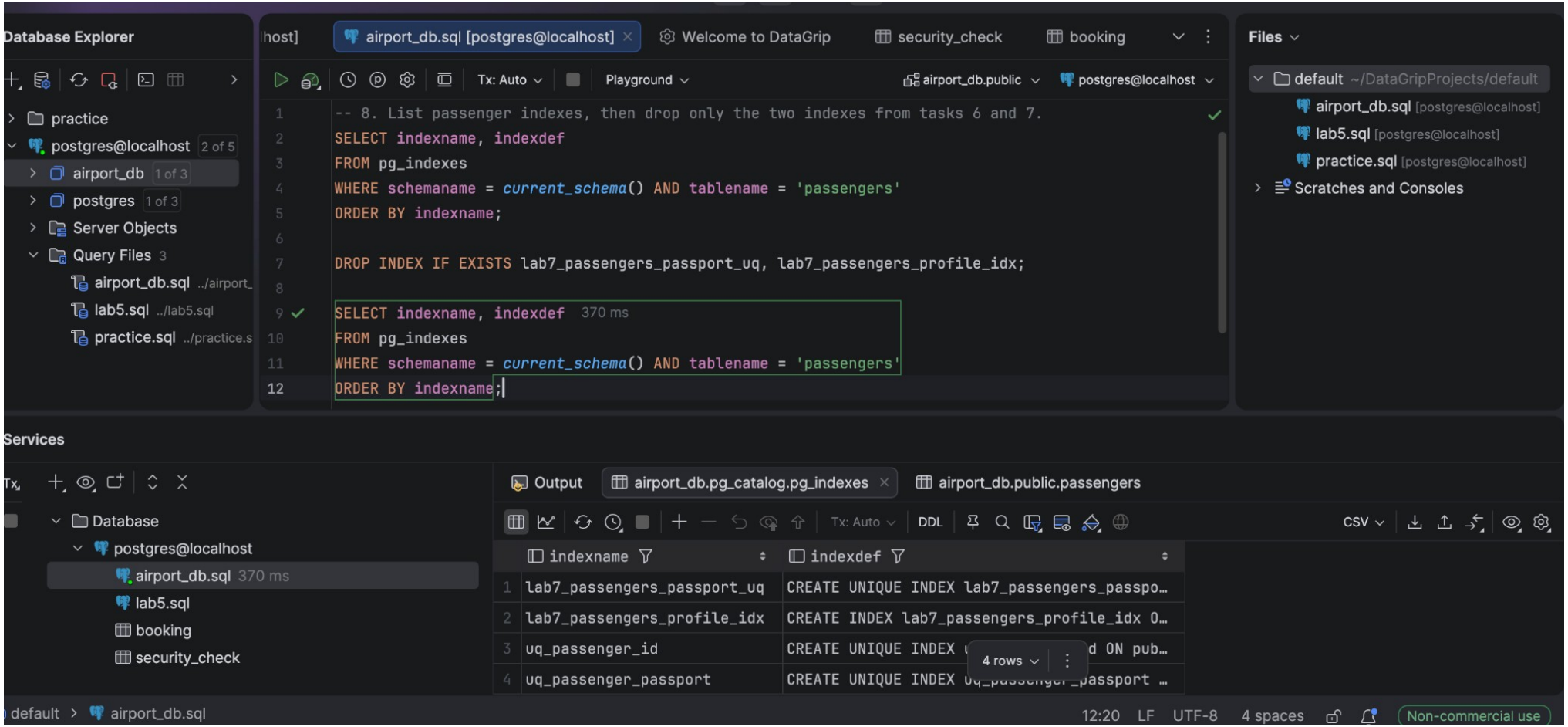
Result and explanation The unique index prevents two passengers from sharing a non-NULL passport number. The INSERT uses two distinct passport numbers. RETURNING shows newly inserted rows; existing passenger IDs are skipped on a rerun.

Task 7 Passenger profile index



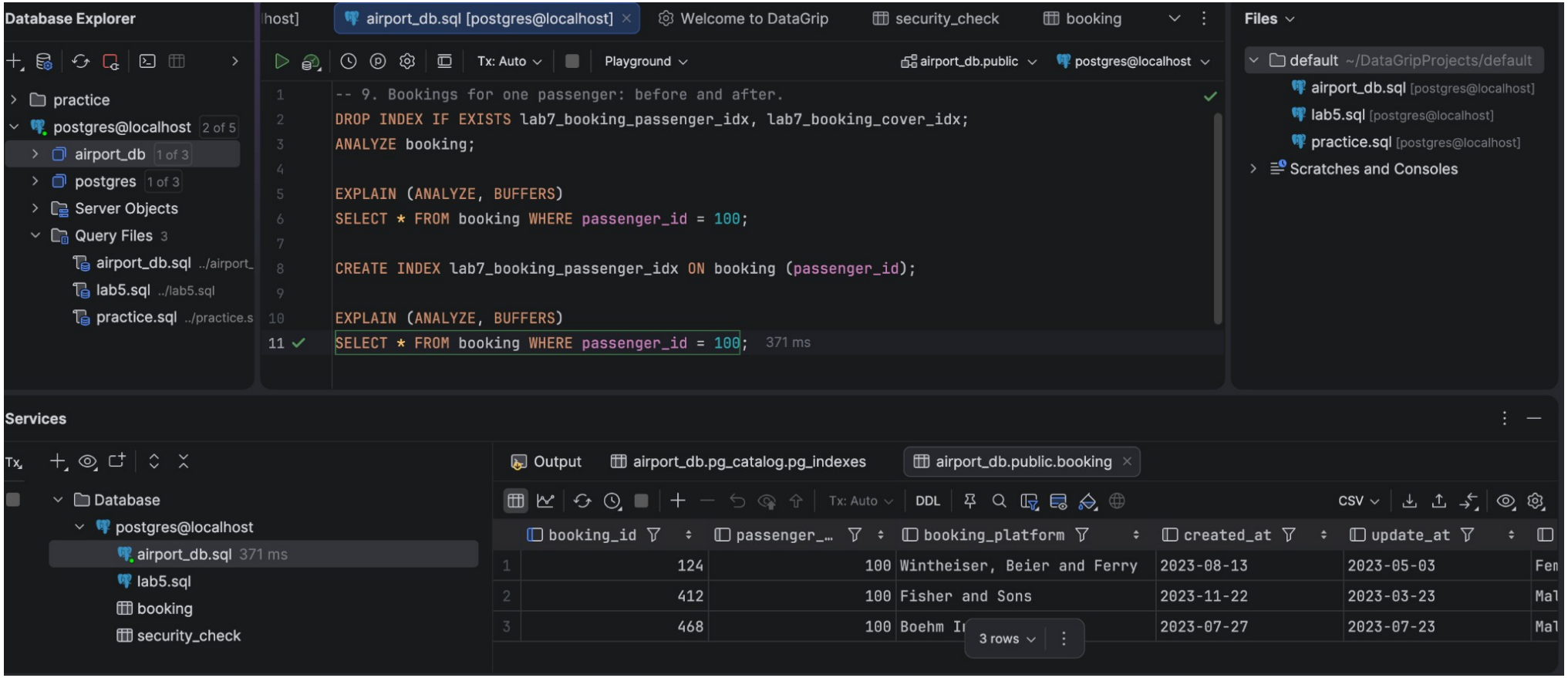
Result and explanation The query finds Howard Igo, born on 1984-03-29, with Philippine citizenship. Citizenship is the available field. The query does not filter on the two leading name columns of the index, so a sequential scan may be cheaper.

Task 8 List and remove passenger indexes



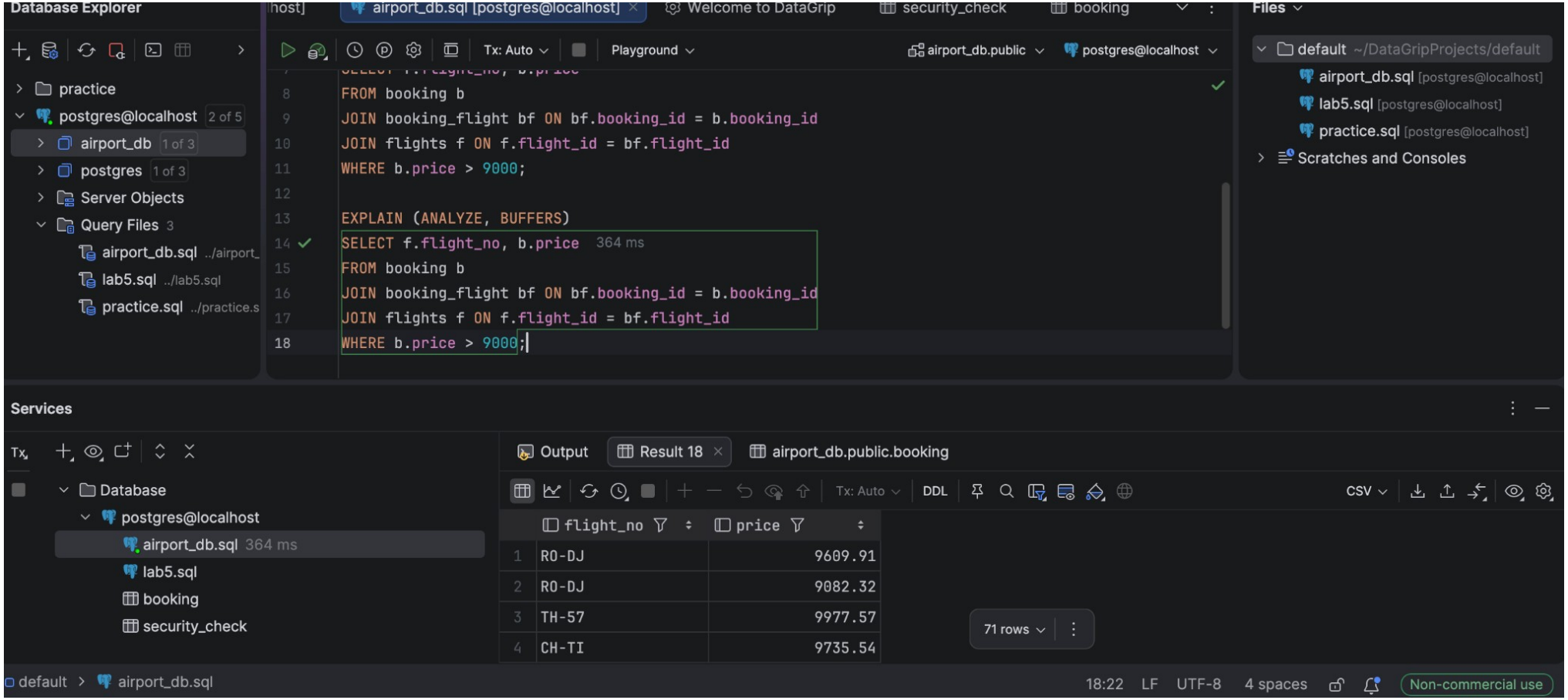
Result and explanation The visible catalog grid lists four indexes, including the two lab indexes. The DROP INDEX statement targets only the lab indexes; the two pre-existing indexes are outside its scope.

Task 9 Bookings before and after indexing



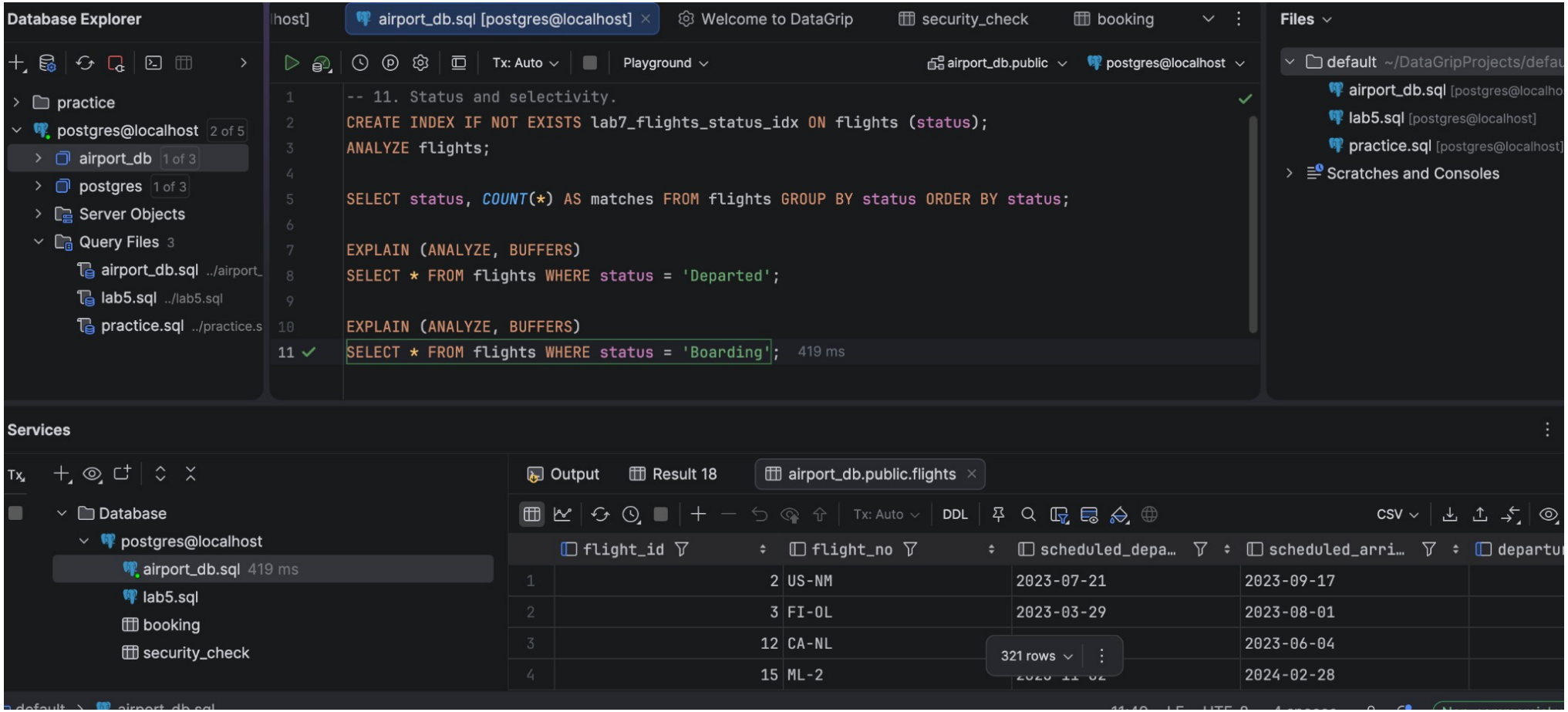
Result and explanation The query returns three bookings for passenger 100. An index on passenger_id can locate those matches directly. The screenshot shows rows, so the scan types and execution times must be read from the two EXPLAIN plans.

Task 10 Join bookings to flights



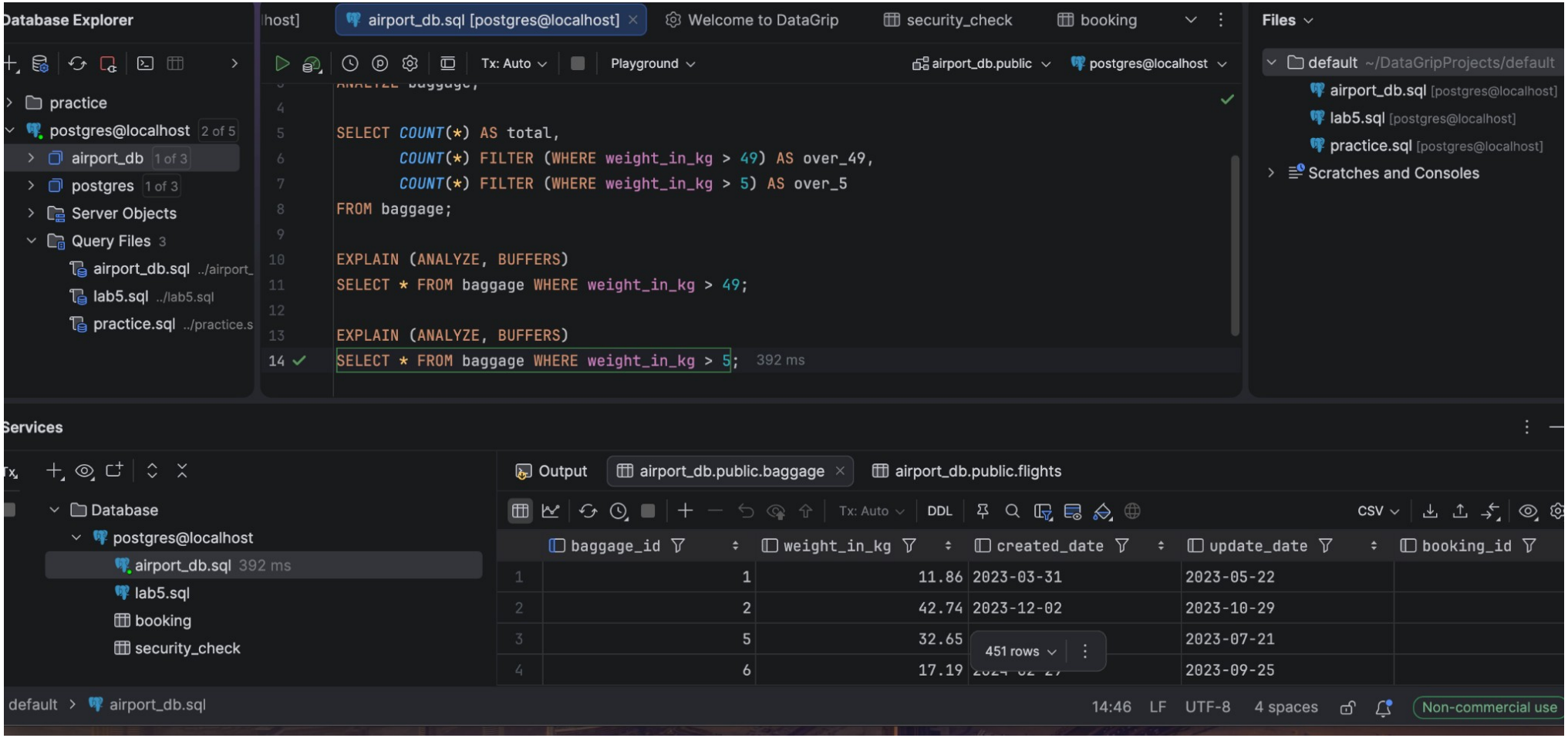
Result and explanation The join returns 71 flight and price rows for bookings priced over 9000. The composite index can support lookups by booking_id. The result grid does not show which access paths the join plan used.

Task 11 Flight status and selectivity



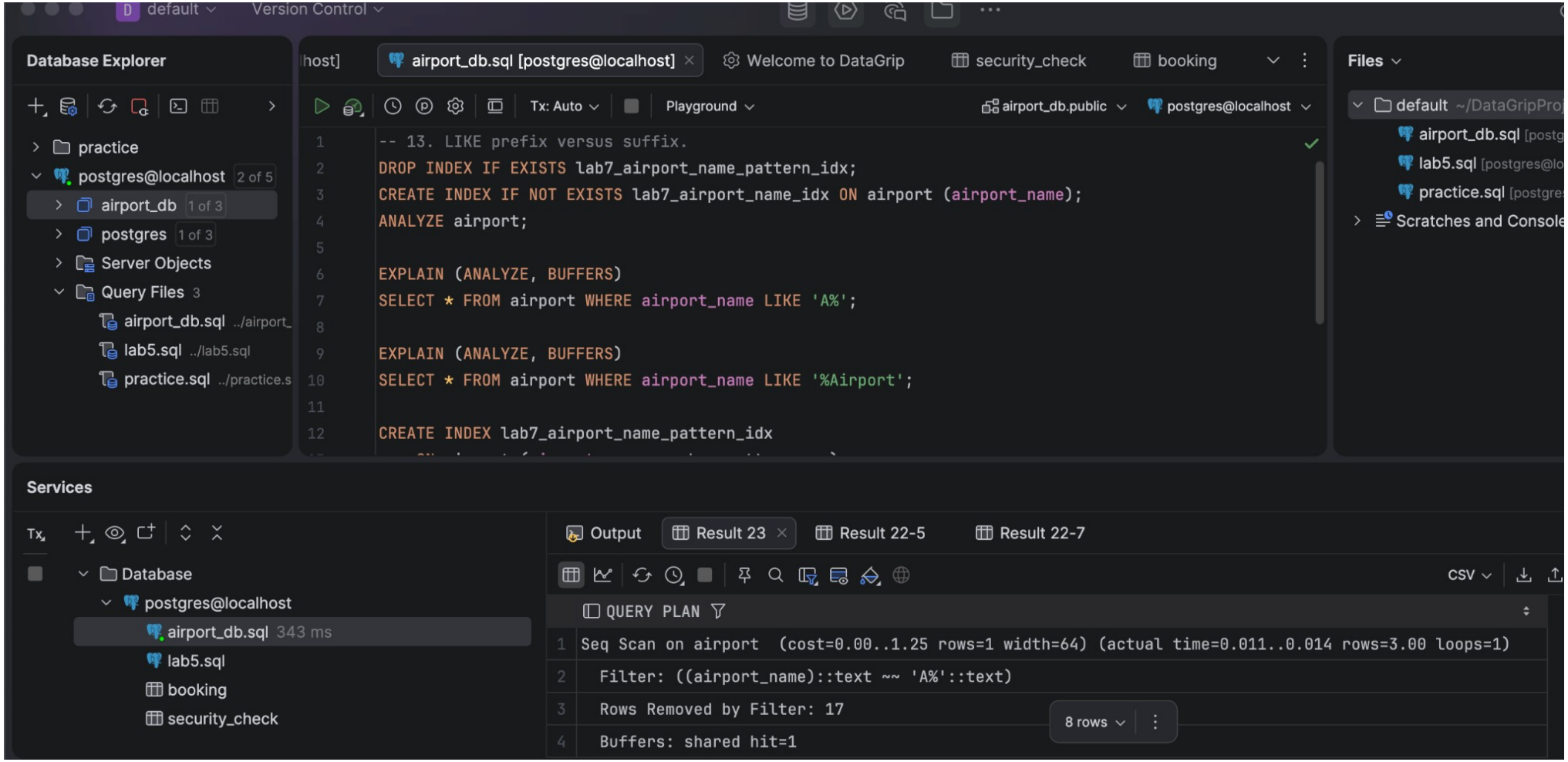
Result and explanation The displayed Boarding query returns 321 rows. The usefulness of the status index depends on selectivity: a rare status can benefit from indexed lookup, while a common status can make a sequential scan cheaper.

Task 12 Baggage weight ranges



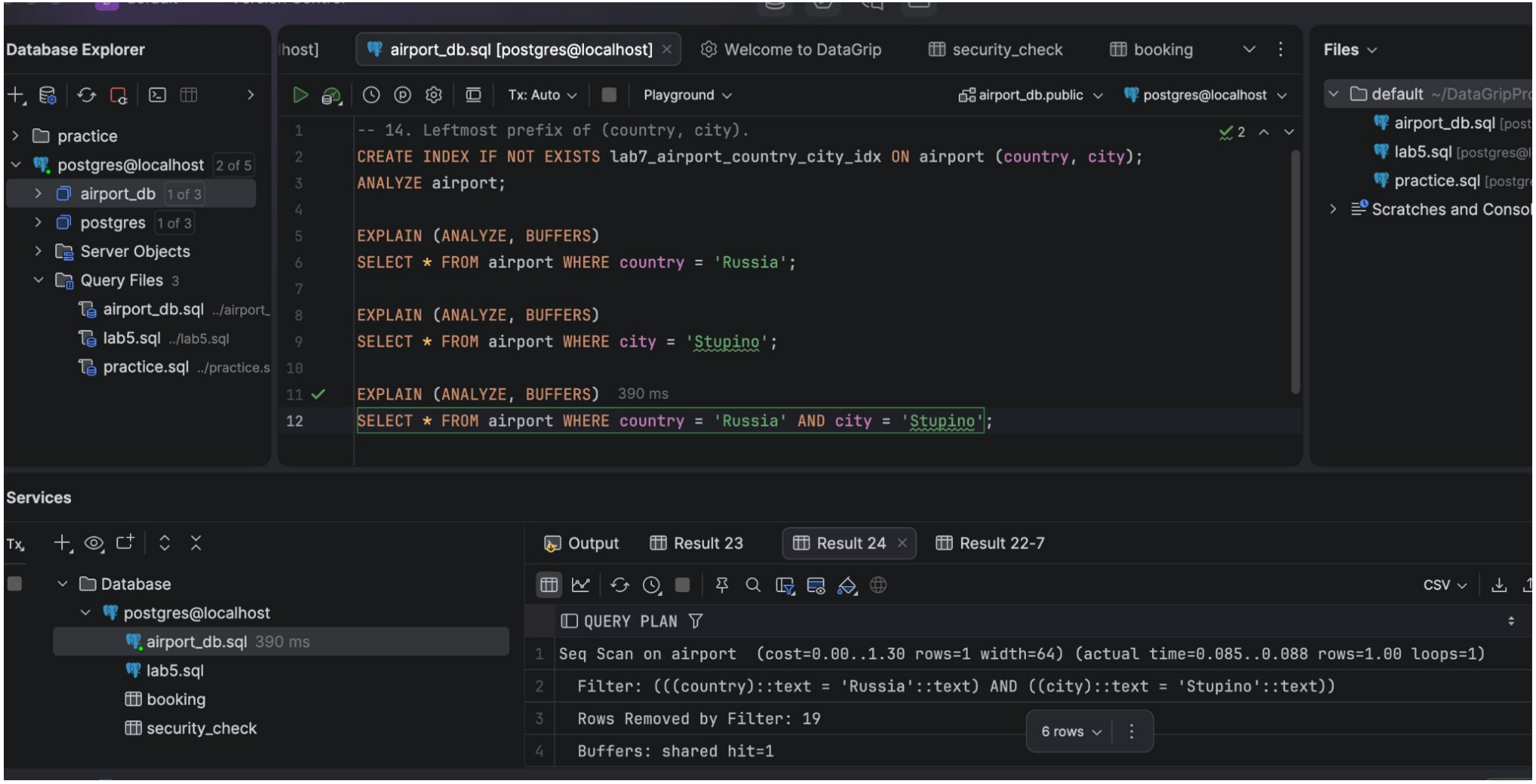
Result and explanation The broad range above 5 kg returns 451 rows in the displayed grid. A narrow range above 49 kg is a better candidate for the weight index because it requires fewer row lookups.

Task 13 Airport name prefix and suffix searches



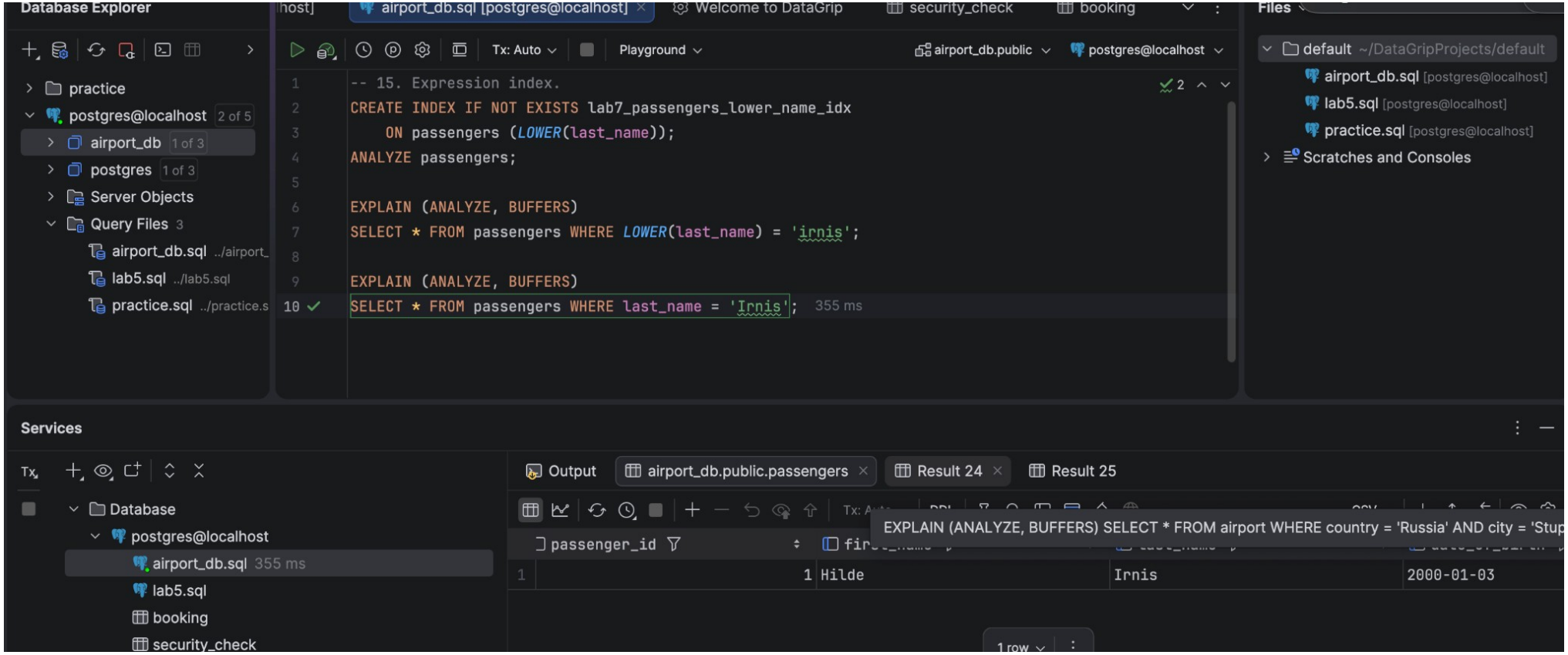
Result and explanation The shown A% plan uses Seq Scan and returns three matches. A fixed prefix can support a B-tree range with appropriate ordering; %Airport has no fixed prefix. On this small table, scanning all rows can still cost less.

Task 14 Leftmost prefix of a composite index



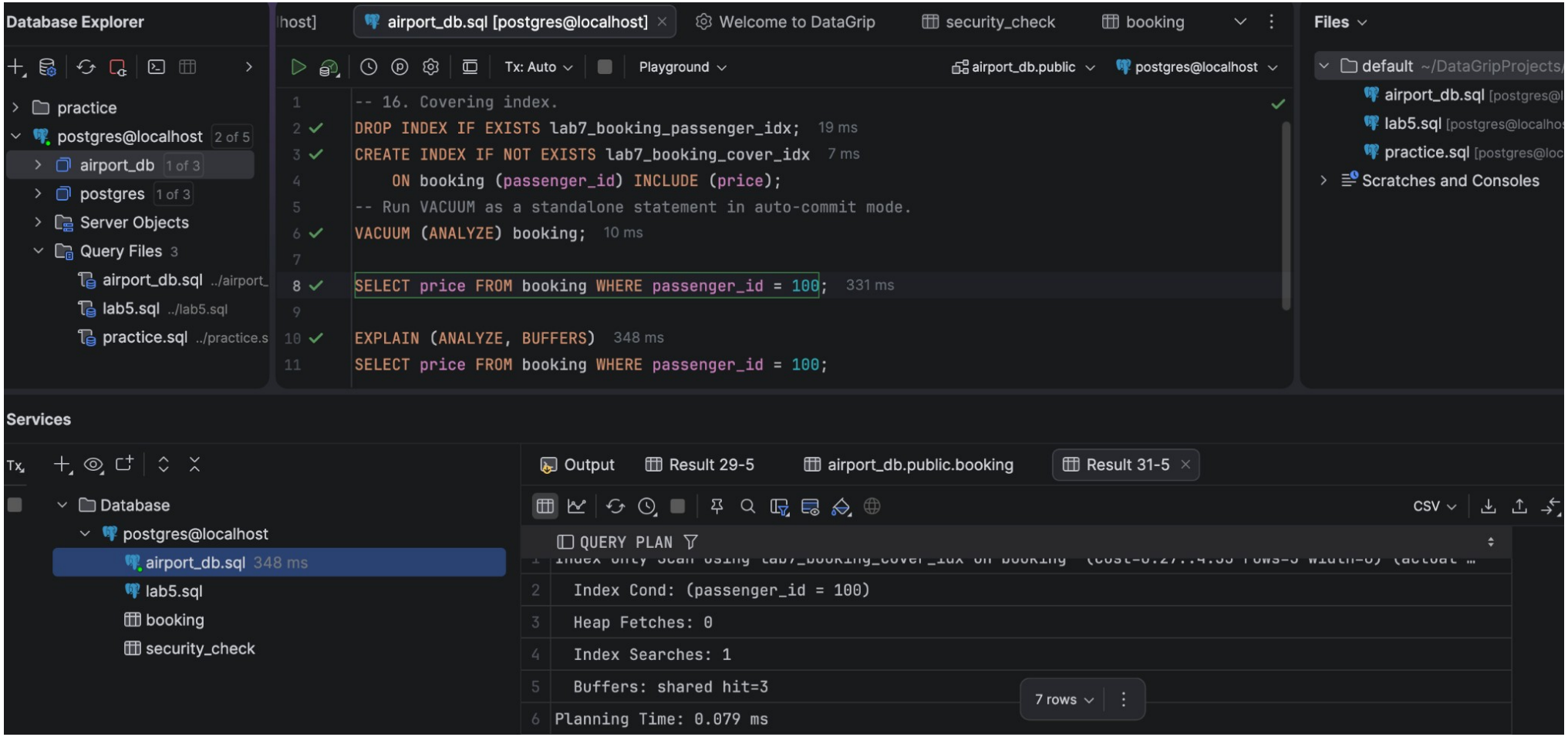
Result and explanation The shown country-and-city query uses Seq Scan and finds one match in a 20-row table. In a (country, city) index, filtering on country constrains the leading key; filtering only on city generally does not narrow the search as effectively.

Task 15 Expression index on last name



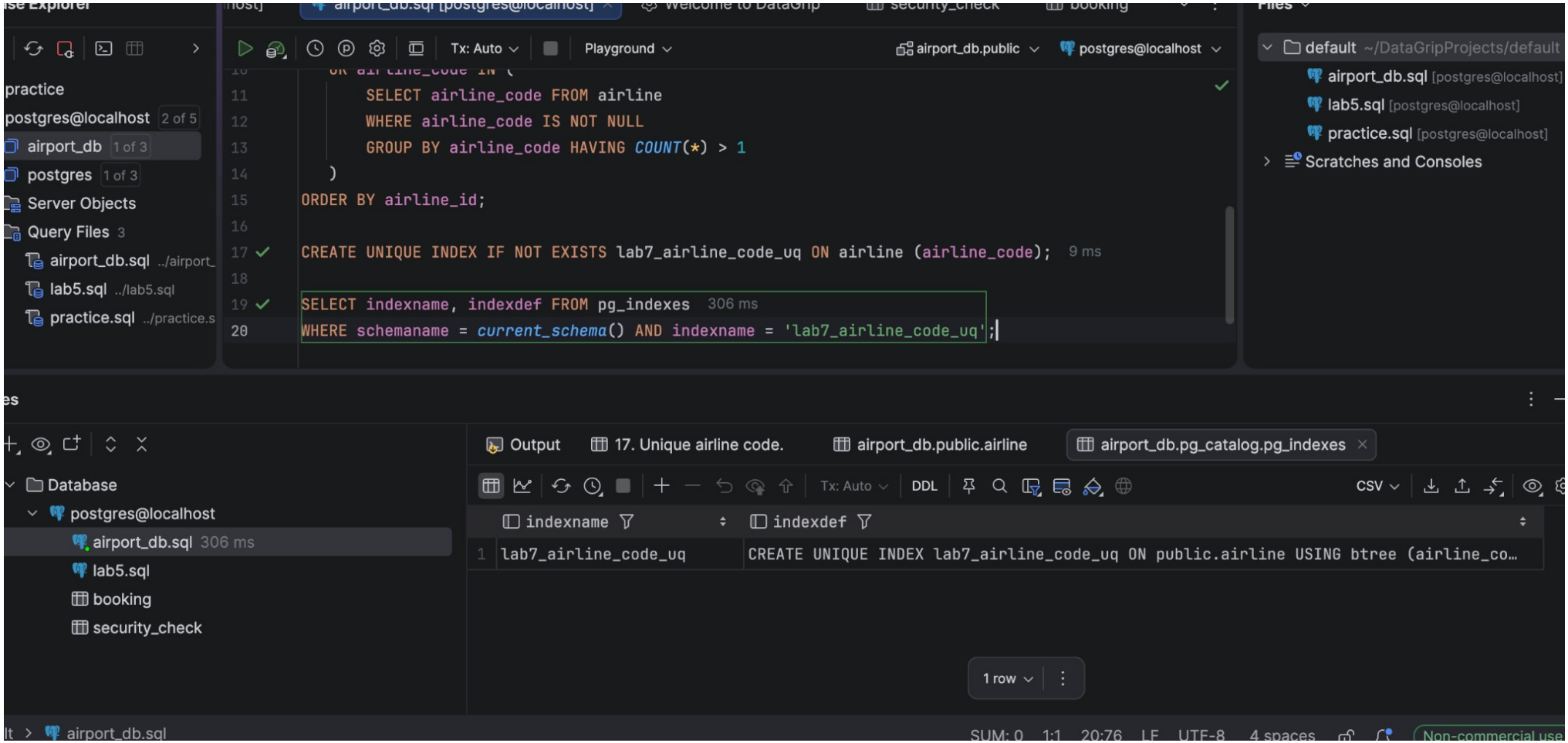
Result and explanation The expression index matches LOWER(last_name) = 'irnis'. A plain last_name = 'Irnis' predicate does not match the indexed expression. The displayed result is Hilde Irnis.

Task 16 Covering index for booking prices



Result and explanation The plan shows Index Only Scan with Heap Fetches: 0. The index supplies passenger_id for the filter and price for the result, so this run did not fetch table rows.

Task 17 Unique airline code



Result and explanation The catalog result confirms that lab7_airline_code_uq was created. PostgreSQL's default unique index rejects repeated non-NULL codes but permits multiple NULL values.